

ÜBUNG 5

Memory & Files

Max Schrötter
schroetter@cs.uni-potsdam.de

Institute for Computational Science
University of Potsdam

19. Dezember 2025



UNTERSCHIED SEMAPHORE UND MUTEX

Vorlesung:

Binäre Semaphore werden auch als Mutex bezeichnet als Kurzform für Mutual Exclusion.



Mutex $\neq$ binäre Semaphore! !

Mutexe besitzen eine Besitzzuordnung (Ownership), Semaphore nicht!

UNTERSCHIED SEMAPHORE UND MUTEX

Semaphore:

```
sem_t s1; // this semaphore is initialized to 0
void client(){
    // do work
    sem_post(&s1);
}
void server(){
    sem_wait(&s1); // waits for client to finish work
    // do work
}
```

UNTERSCHIED SEMAPHORE UND MUTEX

Mutex:

Mutex Type	Robustness	Relock	Unlock When Not Owner
NORMAL	non-robust	deadlock	undefined behavior
NORMAL	robust	deadlock	error returned

- Robust Mutex ist ein Attribut welches dafür sorgt, dass ein Mutex **Fehler** liefert, wenn der Besitzerprozess **abstürzt**.
- Relock: Wenn ein Prozess versucht, einen Mutex zu sperren, den er bereits besitzt.
- Unlock When Not Owner: Wenn ein Prozess versucht, einen Mutex freizugeben, den er nicht besitzt.

AGENDA

1. Seitenersetzungsstrategien

2. Hinweise Blatt 5



OPTIMAL ALGORITHM

- Die Seite mit dem **größten Vorwärtsabstand** wird ausgelagert.

Vorwärtsabstand der **Seite** x zum **Zeitpunkt** t eines Programms mit **Laufzeit** T :

$$d(x, t) := \begin{cases} k, & \text{falls } x \notin \{s[t+1], \dots, s[t+k-1]\} \text{ und } s[t+k] = x \\ \infty, & \text{falls } x \notin \{s[t+1], \dots, s[T]\} \end{cases}$$

W	0	1	2	3	0	1	4	0	1	2	3	4
Rahmen: 0	0	0	0									
1	-	1	1									
2	-	-	2									
Seitenfehler	*	*	*									

- Achtung:** nur für Beurteilung anderer Verfahren interessant, denn $d(x, t)$ nicht vorher bekannt

LRU

- Least Recently Used

W	0	1	2	3	0	1	4	0	1	2	3	4
Rahmen: 0	0	0	0									
1	-	1	1									
2	-	-	2									
Seitenfehler	*	*	*									

WORKING SET

- Working Set der **Größe** $T = 1$
- eine **beliebige Seite** die nicht im Working Set ist, wird ausgelagert:

W	0	1	2	3	0	1	4	0	1	2	3	4
Rahmen: 0	0	0	0									
1	-	1	1									
2	-	-	2									
Seitenfehler	*	*	*									

AGENDA

1. Seitenersetzungsstrategien

2. Hinweise Blatt 5



AUFGABE 1: POSIX-THREADS, READER/WRITER-LOCKS

Programmieraufgabe

1. Gegeben ist eine Implementierung einer **doppelt verketteten** Liste. Erweitern Sie diese um **pthread_rwlock_t**, sodass Sie von mehreren Threads **gleichzeitig** genutzt werden kann!

 `man 3 pthread_rwlock_init` bzw.
`man 3 pthread_rwlock_destroy`
(destroy and initialize a read-write lock object)

 `man 3 pthread_rwlock_rdlock`
(lock a read-write lock object for reading)

 `man 3 pthread_rwlock_wrlock`
(lock a read-write lock object for writing)

AUFGABE 1: POSIX-THREADS, READER/WRITER-LOCKS

Programmieraufgabe

Folgende Fragen sollten bei der Bearbeitung berücksichtigt werden:

- Sollte das Lock eine **globale Variable** oder ein **Teil der Datenstruktur** sein?
- Welche Operation (Lesen oder Schreiben) wird bei der **Standardimplementation bevorzugt**?

AUFGABE 2: BUDDY-SYSTEM

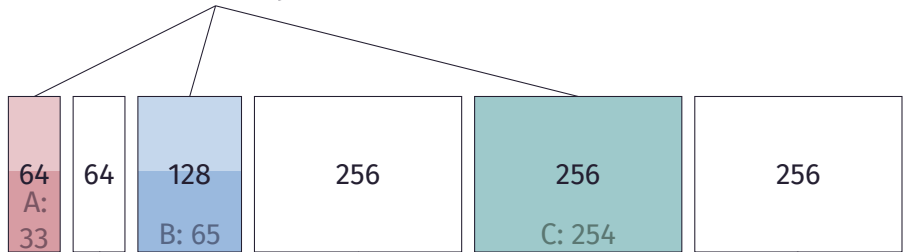
Beispiel:

- Gesamtspeicher 1 GiB, kleinster Block 16 MiB
- A fordert 33 MiB an
- B fordert 240 MiB an
- C fordert 254 MiB an
- D fordert 65 MiB an
- B wird freigegeben
- A wird freigegeben
- E fordert 260 MiB an
- D wird frei gegeben
- C wird freigegeben

siehe Tafel...

AUFGABE 2: SPEICHERVERWALTUNG

Interne Fragmentierung (Verschnitt)
innerhalb der Speicherblöcke



Externe Fragmentierung
des vorhandenen freien Speichers

AUFGABE 3&4: SEITENERSETZUNGSVERFAHREN

Programmieraufgabe

Folgende Fragen sollten bei der Bearbeitung berücksichtigt werden:

- Welches Verfahren führt zu einer **minimalen** Anzahl an Seitenersetzungen?
- Was ist ein **gutartiger** Paging-Algorithmus (siehe Vorlesung)?

Noch Fragen?

Max Schrötter

Potsdam, 19. Dezember 2025

schroetter@cs.uni-potsdam.de